# Graph-Based Recommendations

Dr Seungmin Jin
sedzhin@hse.ru

HSE

# Course Objectives

- Understand graphs, random walks, and PageRank.
- Personalize recommendations using restart distributions.
- Build a PPR recommender
- Compare PPR with MF and discuss their business value.

# Graphs and Bipartite Graphs

Nodes represent entities. Edges represent relationships.

## Graph building blocks



Node: a user or an item



Edge: an interaction or relationship

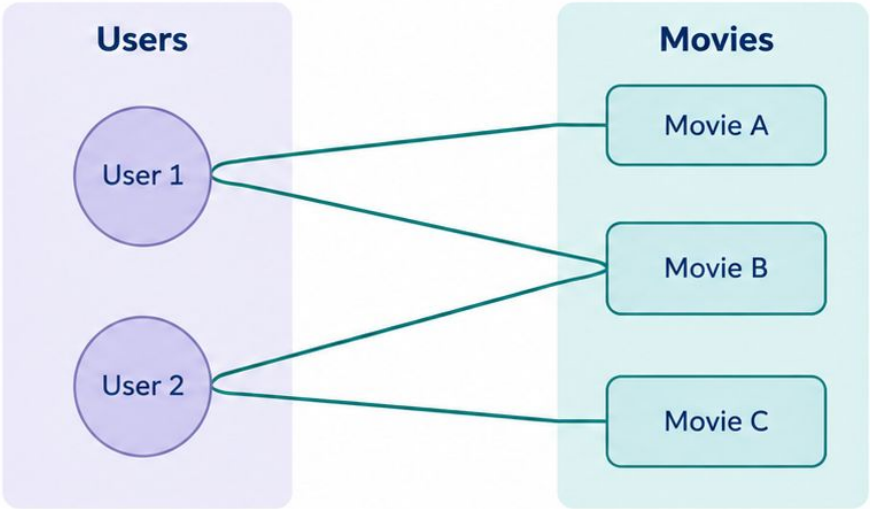


Direction: where a connection points




Weight: strength of a connection

## A user–movie bipartite graph




Edges indicate likes or views

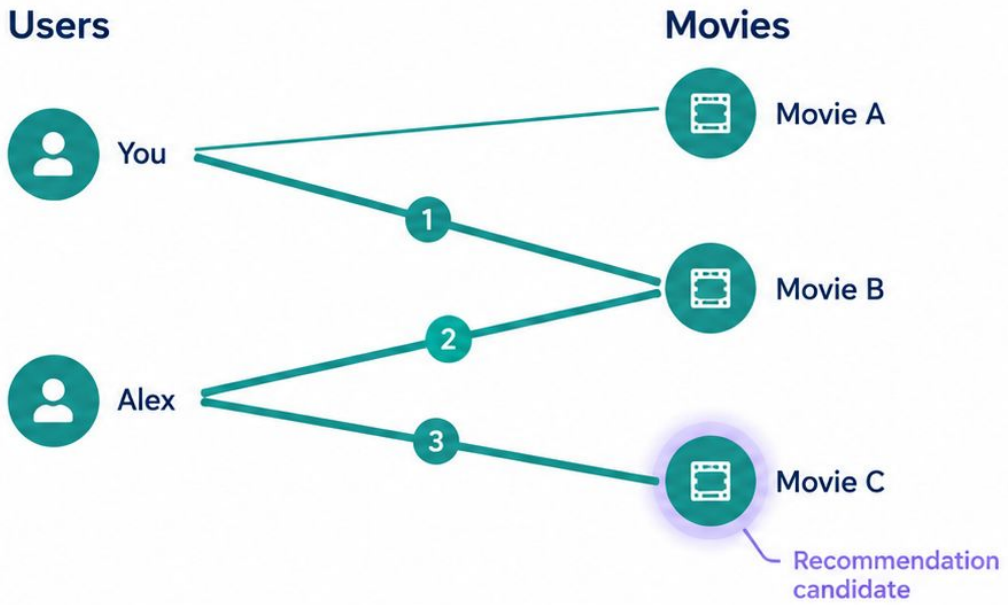
For this example, traversal is allowed in both directions.

**Two distinct node sets. Connections only between the sets.**

User → Movie → User → Movie when following these edges.

# Why Recommend with Graphs?

Turn connections into personalized recommendations.




## What should we recommend next?

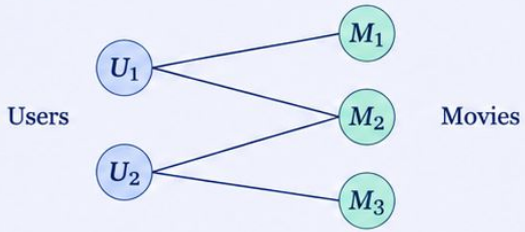
1. Represent interactions as connections
2. Explore paths from the user
3. Rank unseen items by relevance

**Same network. Different starting users. Different recommendations.**

# From a Graph to Matrices

Record connections, combine node types, then normalize each row.

## 1. Interaction matrix $R$




$R$ $(2 \times 3)$

| | $M_1$ | $M_2$ | $M_3$ |
|---|---|---|---|
| $U_1$ | 1 | 1 | 0 |
| $U_2$ | 0 | 1 | 1 |

Rows: users · Columns: movies

## 2. Adjacency matrix $A$

$$A = \begin{bmatrix} 0 & R \\ R^T & 0 \end{bmatrix}$$

(user-user block) (user-movie block) (movie-user block) (movie-movie block)

$A$ $(5 \times 5)$

| | $U_1$ | $U_2$ | $M_1$ | $M_2$ | $M_3$ |
|---|---|---|---|---|---|
| $U_1$ | 0 | 0 | 1 | 1 | 0 |
| $U_2$ | 0 | 0 | 0 | 1 | 1 |
| $M_1$ | 1 | 0 | 0 | 0 | 0 |
| $M_2$ | 1 | 1 | 0 | 0 | 0 |
| $M_3$ | 0 | 1 | 0 | 0 | 0 |

Two node types, one state space

## 3. Transition matrix $P$

$$P_{ij} = \frac{A_{ij}}{\sum_k A_{ik}}$$

$P$ $(5 \times 5)$

| | $U_1$ | $U_2$ | $M_1$ | $M_2$ | $M_3$ |
|---|---|---|---|---|---|
| $U_1$ | 0 | 0 | 0.5 | 0.5 | 0 |
| $U_2$ | 0 | 0 | 0 | 0.5 | 0.5 |
| $M_1$ | 1 | 0 | 0 | 0 | 0 |
| $M_2$ | 0.5 | 0.5 | 0 | 0 | 0 |
| $M_3$ | 0 | 1 | 0 | 0 | 0 |

Every row sums to **1**

**Row = current node. Column = next node.**

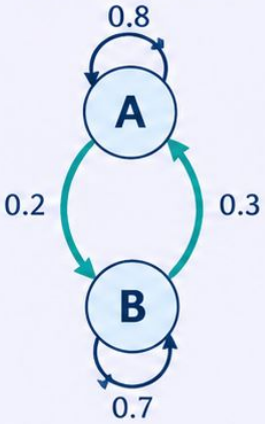
Example: from $U_1$, move to $M_1$ or $M_2$ with probability 0.5 each.

This example has no dangling nodes. Restart is introduced next.

# Random Walks and Probabillity Propagation

One step of a Markov chain moves probability between states.

## 1. Read the transition rules




$\boldsymbol{P}$:

| | A | B |
|---|---|---|
| A | 0.8 | 0.2 |
| B | 0.3 | 0.7 |

Row: current state · Column: next state

## 2. Move probability one step

$$\boldsymbol{p}^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

(*A*, *B*)
Start entirely at A

$$\boldsymbol{p}^{(k+1)} = \boldsymbol{P}^T \boldsymbol{p}^{(k)}$$

$$\boldsymbol{p}^{(1)} = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}$$

A: 80%

B: 20%

## 3. Why the transpose?

$\boldsymbol{P}$ stores outgoing probabilities in rows.

$\boldsymbol{p}$ is a column vector.

$\boldsymbol{P}^T$ collects incoming probability for each destination.

$$p_A^{(k+1)} = 0.8 p_A^{(k)} + 0.3 p_B^{(k)}$$

$$p_B^{(k+1)} = 0.2 p_A^{(k)} + 0.7 p_B^{(k)}$$

*The transpose changes the layout—not the direction of movement.*

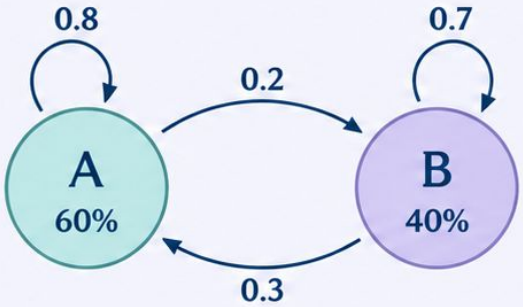
**Next-state probability = sum of incoming probability flows.**

Convention: row-stochastic $\boldsymbol{P}$ and column-vector $\boldsymbol{p}$. Each row of $\boldsymbol{P}$ sums to 1.

# Stationary Distribution

A probability distribution that stays unchanged after one transition.

## 1. Same transition rules




$P =$

| | A | B |
|---|---|---|
| A | 0.8 | 0.2 |
| B | 0.3 | 0.7 |

Rows: current state · Columns: next state

## 2. Verify one step

$$\pi = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

$$P^T\pi = \pi$$

A: $0.8 \times 0.6 + 0.3 \times 0.4 = 0.6$

B: $0.2 \times 0.6 + 0.7 \times 0.4 = 0.4$

**Before:** A 60% · B 40%




**After:** A 60% · B 40%




## 3. Movement continues

A → B: $0.6 \times 0.2 = 0.12$

B → A: $0.4 \times 0.3 = 0.12$

Equal probability flows keep the proportions unchanged in this two-state example.

**Stationary does not mean the walker stops.**

Individual walkers move; the overall distribution stays the same.

**Stationary distribution: $\pi = P^T\pi$, with $\pi_A + \pi_B = 1$.**

For this chain, repeated transitions converge to A 60% and B 40%.

# Two Different Meanings of ‘Settling Down’

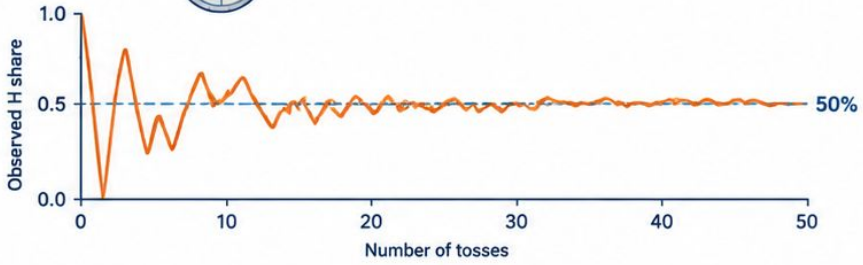
Observed frequencies approach a distribution. A stationary distribution is preserved by a transition.

## 1. OBSERVED FREQUENCIES

**Repeated independent draws**




H T H H T ...




The next-toss probability is ALWAYS 50%.
The recorded fraction approaches 50% as trials accumulate.

Law of large numbers

**Fair die, relabeled faces**

| 1 | 2 | 2 | 3 | 3 | 3 |
|---|---|---|---|---|---|

Long-run recorded shares: 1/6, 2/6, 3/6

## 2. STATIONARY DISTRIBUTION

**One more transition preserves the distribution**




**Fixed transition rules**

| From | To café | To bookshop |
|---|---|---|
| Café | 80% | 20% |
| Bookshop | 30% | 70% |

Next café: 0.60 × 0.80 + 0.40 × 0.30 = 0.60
Next bookshop: 0.60 × 0.20 + 0.40 × 0.70 = 0.40

$$\pi P = \pi$$

**No long observation record is needed to define stationarity.**

## CONNECTED, BUT NOT THE SAME

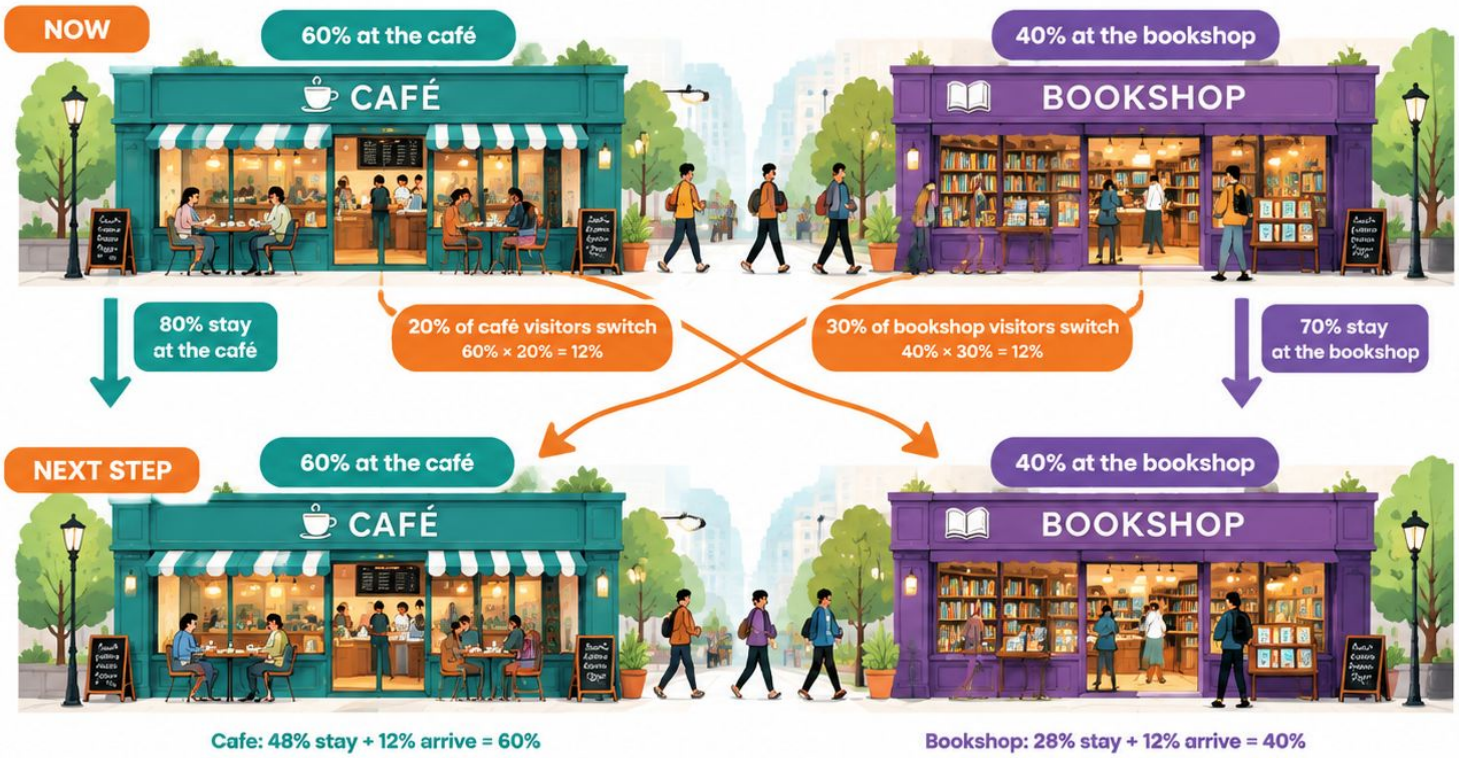
**Frequency convergence: a property of accumulated observations.**
**Stationarity: a property of a probability distribution and transition rule.**

For independent coin tosses, (1/2, 1/2) is stationary from the first toss; only the empirical frequencies need many tosses to stabilize.

Conceptual frequency plot; not measured data.

# People Move. The Propportions Stay.

## Understanding a stationary distribution with two shops




### TRANSITION PROBABILITY

**If I am in this shop, where do I go next?**

A movement rule.

### NODE PROBABILITY

**Where is a randomly chosen person now?**

The share at each shop.

**Stationary ≠ nobody moves.**
**Equal flows keep the overall proportions unchanged.**

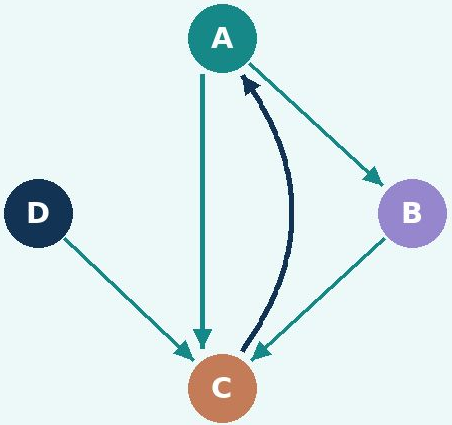
Illustrative expected proportions; actual random headcounts fluctuate. The same group moves only between these two shops.

# PageRank: Follow Links—or Jump

An early Google Search idea: long-run visit probability becomes an importance score.

## 1. Actual web links

Arrow = a link from one page to another




**A → B, C** **B → C**

**C → A** **D → C**

## 2. At every step, choose ONE action

**Currently at A**

**85%**

**Follow a link**

B or C: 1/2 each

B C

Only linked destinations

**15%**

**Random jump**

Any page: 1/4 each

A B C D

No link required

**A jump may return to A itself.**

15% is shared across all four pages.

## 3. Repeat to get a ranking

**Repeat the two-action rule.**

**Measure the long-run probability of each page.**

**Higher visit probability**

**→ Higher PageRank**

**Why add jumps?**

To escape closed link groups.

Jumps are not new web links.

$$p^{(k+1)} = \alpha P^T p^{(k)} + (1 - \alpha) v$$

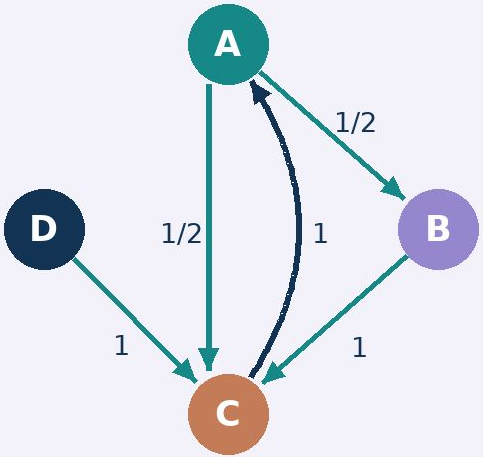
$\alpha = 0.85$ · $v = [1/4, 1/4, 1/4, 1/4]^T$

Example: jump probability to D = 0.15 × 1/4 = 0.0375. No dangling nodes in this graph.

# PageRank in Action: Scores Settle into a Ranking

Same four-page graph · Follow links 85% · Restart 15% · Uniform restart

## 1. Start equally




**A → B, C** **B → C**
**C → A** **D → C**

Start: 0.25 per page

## 2. Repeat the update

| Iteration | A | B | C | D |
|---|---|---|---|---|
| 0 | 0.2500 | 0.2500 | 0.2500 | 0.2500 |
| 1 | 0.2500 | 0.1437 | 0.5687 | 0.0375 |
| 2 | 0.5209 | 0.1437 | 0.2978 | 0.0375 |
| 5 | 0.3885 | 0.2026 | 0.3714 | 0.0375 |
| 10 | 0.3751 | 0.1949 | 0.3925 | 0.0375 |
| 20 | 0.3725 | 0.1958 | 0.3942 | 0.0375 |
| **Limit** | **0.3725** | **0.1958** | **0.3941** | **0.0375** |

**First update for C:**

0.85 × (0.25/2 + 0.25 + 0.25)
+ 0.0375 = 0.56875

## 3. Final ranking

**1. C 39.41%**

**2. A 37.25%**

**3. B 19.58%**

**4. D 3.75%**

**D has no incoming links.**

Restart alone gives D:
0.15 × 0.25 = 0.0375

$$p^{(k+1)} = 0.85\, P^{T} p^{(k)} + 0.15\, v$$

$v = [0.25, 0.25, 0.25, 0.25]^{T}$ · Final ranking: C > A > B > D

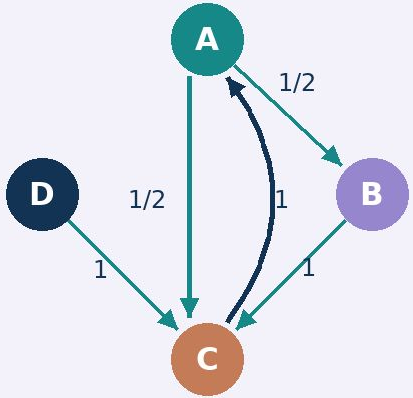
Edge labels: probabilities conditional on following links. Restart arrows omitted.

# Personalized PageRank: Change the Restart, Change the Perspective

Same graph P. Same α = 0.85. Only the restart distribution changes.

## 1. Global PageRank

**Restart → any node equally**

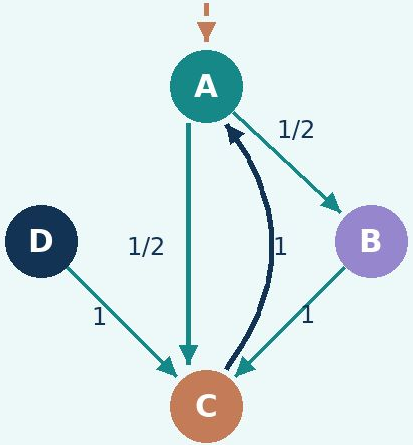



v = [0.25, 0.25, 0.25, 0.25]ᵀ

**One shared ranking.**

## 2. Personalized PageRank

**Restart → A**




v_A = [1, 0, 0, 0]ᵀ

**A is the interest seed—not a new link.**

## 3. Compare the scores

Computed stationary probabilities

| Node | Global | A-seeded |
|---|---|---|
| A | 37.25% | 45.22% |
| B | 19.58% | 19.22% |
| C | 39.41% | 35.56% |
| D | 3.75% | 0.00% |

**Global: C > A > B > D**

**A-seeded: A > C > B > D**

D = 0 here: no links lead to D,
and personalized restart never selects D.

$$p_u^{(k+1)} = \alpha\, P^T\, p_u^{(k)} + (1-\alpha)\, v_u$$

For recommendations: restart at the user, then score movie nodes.

Personalization does not guarantee removal of popularity bias. Edge labels exclude restart.

# Personalized PageRank: Change One Setting at a Time

Same four-node graph throughout. Compare each experiment with the baseline.

Fixed links: A → B, A → C, B → C, C → A, D → C.  A splits link-following probability equally.

## Change the seed

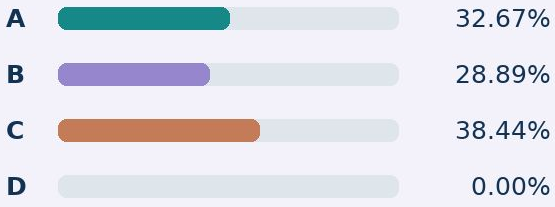
**Only v changes: A → B**

**Restart at B · α = 0.85**

Follow links: 85% | Restart: 15%

$v_B = [0, 1, 0, 0]^T$

**Stationary scores**




**Ranking: C > A > B > D**

Same α as baseline. Different seed.

## Baseline

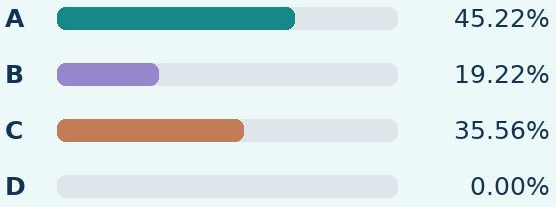
**Reference for BOTH comparisons**

**Restart at A · α = 0.85**

Follow links: 85% | Restart: 15%

$v_A = [1, 0, 0, 0]^T$

**Stationary scores**




**Ranking: A > C > B > D**

α = link-following probability.

## Change α only

**Only α changes: 0.85 → 0.50**

**Restart at A · α = 0.50**

Follow links: 50% | Restart: 50%

$v_A = [1, 0, 0, 0]^T$

**Stationary scores**




**Ranking: A > C > B > D**

Same seed as baseline. More restart.

**Seed v: whose perspective?**  **α: how far to explore before restarting?**

$$p_u^{(k+1)} = \alpha P^T p_u^{(k)} + (1 - \alpha) v_u$$

Lower α means more frequent restart and more weight on short paths from the seed.

A and B are seed nodes in this toy graph—not user/movie labels. Bars use the same scale; values rounded.

# From PPR Scores to Recommendations

Rank eligible products—not every node in the graph.

**Personalized exploration: follow links 85% of the time; restart at User A 15% of the time.**

User A is the restart destination—not a fixed score and not the only node visited.

## 1. Keep product nodes

Illustrative scores—not computed PPR

| Node | Type | Score |
|---|---|---|
| User A | User | 0.40 |
| Product 1 | Product | 0.24 |
| Product 2 | Product | 0.16 |
| Product 3 | Product | 0.12 |
| Product 4 | Product | 0.08 |

Exclude User A: users are not products.

## 2. Exclude ineligible items

Example policy: new, available products

| | |
|---|---|
| **Product 1** | Already purchased |
| **Product 2** | Available / unseen |
| **Product 3** | Out of stock |
| **Product 4** | Available / unseen |

Repurchase tasks may use different rules.

## 3. Select Top-K

K = 2 · Sort by descending score

1. **Product 2**
   PPR score: 0.16
2. **Product 4**
   PPR score: 0.08

**Do not relabel these as purchase odds.**

No renormalization needed for ordering.

## PPR score ≠ purchase probability

A score of 0.16 refers to the model's long-run node occupancy—not a 16% chance of buying.

**Different scoring methods. The same eligibility → ranking → Top-K pipeline.**

### Neighbor-based CF

Similar users or similar items provide preference evidence.

### Matrix factorization (MF)

Learn user/item latent vectors; score with their interaction.

### Personalized PageRank

Propagate scores through paths; restart from a user-specific seed.

MF is a model-based CF method. Additional reranking and purchase-probability modeling are omitted.

# Graph Recommendations in Practice: Pixie and Session Intent

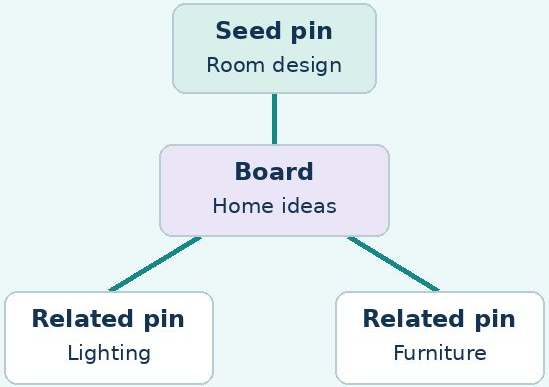
Real-world deployment → a general design limitation → research ideas

## 1. Pixie at Pinterest

**A deployed random-walk recommender**

Pins = content. Boards = collections.

Edges mean “this pin is saved on this board.”




**Explore: pin → board → other pins**

Published in 2018; not today’s full stack.

[1711.07601] Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time

## 2. What matters RIGHT NOW?

**Example: past interests ≠ current intent**

**Long-term history**
Frequently saves interior-design ideas

**Current session**
Now exploring tents and camping gear

If only old interests seed the walk, recommendations may miss today’s goal.

**A general design limitation**

Not a claim that Pixie ignores sessions.
Pixie can use user-specific pin seeds.

## 3. Session-aware ideas

**Research example: S-Walk**

**Recent session items**
Tent → sleeping bag → hiking gear

↓

**Session-based restart**
Adapt where the walk returns

↓

**Item transition relationships**
Use session patterns in exploration

**Idea only: recent context + graph walks.**

Model equations and training are omitted.

## Personalized does not automatically mean aware of current intent.

The walk can reflect “now” only when its seeds or transition rules contain current-context signals.

Sources: Pixie (WWW 2018) · S-Walk: Accurate and Scalable Session-based Recommendation · Examples are illustrative.

# Newest Click—or Relevant History?

**Current search: camping tent**

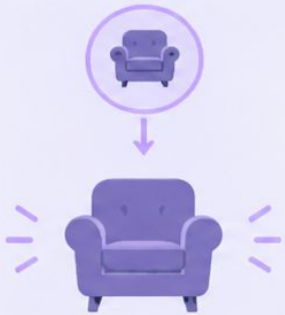
Repeated camping interest

One furniture click

Past months

Just now

## Recency only



**Newest event gets priority**

## Context-aware attention




**Relevant history can matter more**

illustrative example—not measured recommendations.

## Older does not mean irrelevant.

Attention can weigh history using content, time and search context.

# Discussion

# HW5. From Ratings to Recommendations

## 1. Implementation Task

https://dryjins.github.io/RecSys-LLMs/week5/

- Repository: dryjins/RecSys-LLMs/tree/main/week5
- Dataset: MovieLens 100K; use `week5/u.data` for user–movie ratings and `week5/u.item` for movie titles and genre indicators.1453
- Implementation: Build a JavaScript-based application to train and evaluate matrix factorization.
- Data Preparation: Report users, movies, ratings, and matrix sparsity. Treat missing ratings as unobserved, not zeros.
- Data Split: Sort each user's ratings by timestamp; use 80% for training, 10% for validation, and 10% for testing. Report exclusions of users or movies absent from training.
- Baseline: Predict ratings using the training-set mean.
- Matrix Factorization: Learn 16-dimensional user and movie vectors; predict ratings using their dot product.
- Training: Minimize squared error on observed training ratings. Select the checkpoint using validation RMSE; report baseline and MF test RMSE.
- Recommendations: Produce Top-10 lists for two test users, showing movie titles and scores. Exclude movies rated in training or validation.
- Evaluation: Treat test ratings ≥ 4.0 as relevant. Report average Recall@10 over users with eligible relevant test movies, using all training-set movies as candidates.

## 2. Business & Algorithmic Analysis

- **Familiar Choices vs. New Discoveries:** Do your recommendations help users discover movies, or mainly repeat their existing preferences? Compare two users' Top-10 lists with their past ratings. Use specific movies or genres to discuss familiar choices, new discoveries, and their value for a streaming platform.

# HW6. Personalized PageRank Movie Recommendations

## 1. Implementation Task

- Repository: [dryjins/RecSys-LLMs/tree/main/week6]
- Dataset: Reuse MovieLens 100K (`week5/u.data` and `week5/u.item`) and the previous per-user chronological 80%/10%/10% split.
- Graph: Build a user–movie graph from training ratings ≥ 4.0. Treat missing ratings as unobserved.
- Implementation: Compute global PageRank and user-seeded PPR with TensorFlow.js. Define α\alphaα as the link-following probability.
- Verification: Check transition normalization, probability conservation, convergence, and zero-degree-node handling.
- Evaluation: Select PPR's α\alphaα using validation Recall@10. Compare popularity, global PageRank, PPR, and your previous MF on the same eligible test users and candidate movies.
- Filtering: For validation, exclude movies rated in training. For testing, exclude movies rated in training or validation.
- Recommendations: Produce test Top-10 lists for the same two users, showing movie titles, genres, and scores.
- Controlled Comparison: Separately change one setting at a time—user A/B at α=0.85\alpha=0.85α=0.85, then user A at α=0.85\alpha=0.85α=0.85 versus 0.500.500.50. Use the validation-selected α\alphaα for the final PPR evaluation.
- Reporting: Report test Recall@10, evaluation exclusions, and recommendation popularity based on positive training ratings. PPR scores are not predicted ratings or purchase probabilities.

## 2. Business & Algorithmic Analysis

Should the streaming platform keep MF, replace it with PPR, or use the two methods in different recommendation sections? Choose one strategy and justify it using your Recall@10 results and specific movie examples from the same two users' Top-10 lists.